

Card 1: Columnar Row Groups

Slices columns into horizontal Row Groups of 122,880 rows. Contiguous memory layout optimized for scan workloads.

Card 5: In-Memory WAL

Logs insert/delete transactions to append-only disk files. Merges states to storage during checkpointing.

Card 9: Dynamic Casting

Calculates types dynamically, casting mismatch vectors in batch. Employs Selection Vectors to skip filtering copies.

Card 13: Logical Deletes

Marks deletes in a Delete Vector index per Row Group. Cascades update versions using physical Version Chains.

Card 2: Block Allocation

Manages database file using 256KB block allocations. Segment headers track offsets and compression schemas.

Card 6: Vectorized Core

Executes operators on Vectors of 1024 or 2048 values. Contiguous cache-aligned buffers reduce dynamic virtual dispatch costs.

Card 10: Window Function

Stores sliding window partitions in chunk buffers. Calculates aggregates dynamically using pointer jumps.

Card 14: Column Compaction

Reclaims stale versions after Safe Epochs. Compresses datasets using RLE, Bit-Packing or Dictionary coding.

Card 3: Buffer Manager

Pages block segments in/out of memory. Invokes eviction queues and spills operator states on memory limits.

Card 7: Volcano vs Vectorized

Volcano runs row-by-row loops causing high branches. Vectorized processes chunks inside CPU L1 cache boundaries.

Card 11: Transaction Epochs

Implements MVCC using monotonic system Epoch counters. Transactions bind to Read Epochs at startup.

Card 15: ACID Checkpoint

Commit flushes transaction buffers and WAL. Rollbacks discard cache memory structures and invalidate log segments.

Card 4: Zone Map Indexes

Stores Min-Max values in segment metadata. Skips reading irrelevant segments dynamically on queries.

Card 8: SIMD Acceleration

Loops over contiguous vector memories are compiled into SIMD instructions (AVX2/512) for parallel calculations.

Card 12: MVCC Visibility

Rows carry insert_id and delete_id. Evaluates visibility on the fly by comparing Read Epoch without locking.

Card 16: Push-Based Executor

Pushes vectorized chunks from Scan nodes up through execution pipelines, optimizing concurrency scheduling.

Card 17: Parallel Scan

Divides row groups into parallel scan Tasks. Idle threads steal scanning tasks to balance system CPU load.

Card 21: Parquet Zero-Copy

Maps Parquet columns directly to Vector structures. Pushes filters to row group metadata to avoid decompression.

Card 25: Catalog Schema

Manages database metadata namespaces and schemas. Version changes are isolated using Epoch MVCC.

Card 18: In-Memory Hash Join

Builds cache-friendly tables in RAM. Probes matches in vectorized batches using selection arrays.

Card 22: Arrow Integration

Shares data with Apache Arrow formats by mapping memory pointers directly, bypassing serialization costs.

Card 26: UDF Plugins SDK

Exposes C++ extension APIs. Vectorized custom functions process data chunks using compiled SIMD pathways.

Card 19: External Merge Sort

Sorts large tables by spilling sorted runs to temporary files when database memory limits are breached.

Card 23: CSV JSON Scanners

CSV reader splits inputs across threads and runs type inference to compile parsing pathways dynamically.

Card 27: Profiling Diagnostics

Traces operator runtimes, vector counts and disk spilling using json-based profiling pragma commands.

Card 20: Out-of-Core Spill

Memory Manager offloads large aggregation buffers or join tables to disk storage to avoid OOM crashes.

Card 24: Embedded Engine

Links as a C++ library inside host processes. Internal locks protect Catalog and Transactions to secure concurrency.

Card 28: Resource Limits

Enforces memory hard limits via configuration commands, throttling queries to secure host environments.